ARAE - AI 건축 평면도 분석 및 법규 검색 플랫폼

> 건축 평면도를 AI로 분석하고, 법규/조례를 RAG 기반으로 검색하는 풀스택 웹 서비스

항목	내용
기간	약 8주
인원	6명
역할	풀스택 (Frontend + Backend + AI Server + Infra)

프로젝트 배경

건축 평면도 검토는 전문가가 도면을 육안으로 확인하고, 관련 법규를 수동으로 조회하는 방식으로 진행됩니다. 이 과정은 시간이 오래 걸리고 검토자의 숙련도에 따라 결과가 달라지는 문제가 있습니다. ARAE는 CV 모델로 평면도의 공간 구조를 자동 분석하고, RAG 기반 LLM이 건축 메트릭 산출과 법규 검색을 수행하여, 건축 평면도 검토 과정을 자동화합니다.

시스템 아키텍처

```
[React Frontend :3000]  ──  React 19 + TypeScript 5.9 + Vite 7.2
        │ REST API + JWT Auth
[Spring Boot Backend :8080]  ──  Java 21 + JPA + Spring Security + AWS S3 SDK
        │                  │
  [AWS RDS PostgreSQL]   [Python FastAPI :8000]
  [+ pgvector 1024-dim]   ├─ CVAnalysisAgent ── RunPod Serverless (YOLOv5 +
DeepLabV3+)
                          ├─ FloorplanSearchAgent ── RunPod Pod vLLM (Qwen3-8B +
LoRA)
                          └─ RegulationSearchAgent ── OpenAI GPT-4o-mini
```

기술 스택

영역	기술
Frontend	React 19, TypeScript 5.9, Vite 7.2, CSS Modules, Axios, React Router v7
Backend	Spring Boot 3.4, Java 21, JPA, JWT (HS256), Spring Security, AWS S3 SDK
AI Server	FastAPI, YOLOv5, DeepLabV3+, Qwen3-8B (LoRA fine-tuning), GPT-4o-mini
DB	PostgreSQL, pgvector (1024-dim 벡터 검색), Hibernate Vector
Infra	AWS EC2, RDS, S3, RunPod GPU (A100 SXM 80GB), GitHub Actions CI/CD

핵심 기능 (1)

1. 도면 AI 분석 — 2단계 프로세스

4단계 CV 파이프라인(OBJ/OCR/STR/SPA)으로 평면도에서 객체 탐지, 공간명 인식, 구조/공간 세그먼테이션을 수행합니다. Shapely 기반 기하학 처리로 토폴로지 그래프를 생성하고, RAG(pgvector 유사도 검색 + 사내 평가 기준)를 결합한 LLM이 13개 건축 메트릭과 분석 보고서를 생성합니다.

Step 1 — 분석 (미리보기)

```
React → POST /api/floorplan/analyze (이미지)
  → Spring Boot → POST python:8000/analyze
    → CV: YOLOv5(OBJ) → YOLOv5(OCR) → FCN(STR) → DeepLabV3+(SPA)
    → Shapely 기하학 → 토폴로지 그래프 → RAG + LLM 분석
  ← topology_json + topology_image + llm_analysis_json (13개 메트릭)
```

Step 2 — 저장 (메타데이터 생성)

```
React → POST /api/floorplan/save (이미지 + llm_analysis_json)
  → Spring Boot → POST python:8000/generate-metadata
    → 13개 메트릭 추출 + document 생성 + Qwen3-Embedding 임베딩 (1024-dim)
  → S3 이미지 업로드 + DB 저장 (FloorPlan + FloorplanAnalysis)
```

13개 메트릭: 구조 유형(판상형/타워형/혼합형), Bay 수, 방 수, 욕실 수, 무창실 수, 발코니/거실/욕실/주방 비율, 특수 공간 유무, 기타 공간 유무, 환기 품질, 법규 준수 등급

2. 텍스트 기반 도면 검색 챗봇

GPT-4o-mini가 사용자 질문의 의도를 분류(도면 검색/법규 검색)하고 에이전트로 라우팅합니다.

```
React → POST /api/chatbot/chat (question)
  → Spring Boot → POST python:8000/orchestrate
    → OrchestratorAgent._classify_intent() (GPT-4o-mini)
      → FLOORPLAN_SEARCH: FloorplanSearchAgent
        → 하이브리드 RAG (벡터 유사도 0.8 + 키워드 매칭 0.2)
        → sLLM(Qwen3-8B)이 [도면 #N] 마커 포함 구조화 답변 생성
  → Spring Boot: floorplan_ids → DB 조회 → S3 image_urls 변환
React: [도면 #N] 마커를 정규식 파싱 → 요약 텍스트 + 도면 썸네일 카드 표시
```

핵심 기능 (2)

3. 이미지 기반 유사 도면 검색

업로드된 도면을 CV 파이프라인으로 분석한 뒤, 추출된 메트릭을 이산형 필터로 변환하여 pgvector 코사인 유사도 검색을 수행합니다.

```
React → POST /api/chatbot/chat (image)
  → [Step 1] CVAnalysisAgent.execute(mode="full")
    → CV 추론 → LLM 분석 → 13개 메트릭 추출 → 임베딩 생성 (1024-dim)
  → [Step 2] FloorplanSearchAgent.execute(mode="image_search")
    → 메트릭 → 이산형 필터 (structure_type, room_count, bay_count, bathroom_count)
    → pgvector 코사인 유사도 검색 (필터 적용)
    → 결과 부족 시 7단계 필터 점진적 완화 (Progressive Relaxation)
    → sLLM이 [유사 도면 #N] 마커 포함 구조화 답변 생성
React: [유사 도면 #N] 마커 파싱 → 분석 텍스트 + 유사 도면 썸네일 카드
```

필터 점진적 완화 (7단계): 초기 필터(구조 유형 + 방 수 + Bay 수 + 욕실 수)로 검색 후, 결과가 3개 미만이면 욕실 수 → Bay 수 → 방 수 ±1 → 구조 유형 순으로 필터를 점진적으로 완화하여 유사 도면 3개 반환을 보장합니다.

4. 법규/조례 검색

```
사용자 질문 → OrchestratorAgent → REGULATION_SEARCH 판정
  → RegulationSearchAgent
    → 주소, 용도지역, 활동 유형 파싱
    → 토지특성(land_char) + 법규(law) + 건축물용도(useBuilding) DB 조회
    → 벡터 + 텍스트 하이브리드 검색
    → Cross-encoder(ms-marco-MiniLM-L-6-v2) 리랭킹으로 정밀도 향상
    → 임베딩 LRU 캐시(500개)로 중복 호출 방지
```

5. 관리자 대시보드

6개 통계 카드(사용자, 도면, 채팅 등), 도면 DB 관리(검색/필터/일괄삭제/상세모달), 활동 로그(타입/검색/날짜 필터)를 제공하며, 서버사이드 페이지네이션과 이미지 줌/팬을 지원합니다.

데이터베이스 설계

Entity 관계도

```
┌──────────────┐
│   User       │
│  (PK: id)    │
└──────────────┘
      │
      │ (1:N)
      ├──────────────────────────────┐
      ▼                              ▼
┌──────────────┐                  ┌──────────────┐
│  ChatRoom    │ ──(1:N)──┐       │  FloorPlan   │
│ FK: user_id  │          │       │ FK: user_id  │
└──────────────┘          │       └──────────────┘
                          │              │
                          ▼              │ (1:1, cascade ALL)
                  ┌──────────────┐       ▼
                  │ ChatHistory  │   ┌──────────────────┐
                  │ FK: chatroom │   │FloorplanAnalysis│
                  │ - question   │   │FK: floorplan_id  │
                  │ - answer     │   │- 13개 메트릭     │
                  │ - imageUrls  │   │- embedding       │
                  └──────────────┘   │  (vector 1024)   │
                                     └──────────────────┘

독립 테이블 (참조 데이터):
├─ InternalEval  (사내 평가 기준 + embedding vector 1024)
├─ LandChar      (토지특성정보 - 법정동코드, 용도지역, 지목, 면적)
├─ Law           (법규/조례 - 용도지역별 허가 구분)
└─ Usebuilding   (건축물용도 정의 - 카테고리, 시설명, 설명)
```

주요 테이블 상세

테이블	주요 필드	특이사항
users	email(unique), pw(BCrypt), role(USER/ADMIN)	@CreationTimestamp 자동 생성일
floorplan	user_id(FK), imageUrl(S3), assessmentJson	orphanRemoval=true, 삭제 시 연쇄 삭제
floorplan_analysis	13개 메트릭, embedding(vector 1024)	@JdbcTypeCode(SqlTypes.VECTOR)
chathistory	question, answer, imageUrls(JSON 문자열)	@JsonBackReference 순환 참조 방지
internal_eval	keywords, document, embedding(vector 1024)	RAG 사내 평가 기준 검색용
land_char	법정동코드, 용도지역(zone1/2), 지목, 면적	CSV 배치 로드 (JDBC batch, 5,000건)
law	지역코드, 법률명, 용도지역명, 허가구분	CSV 배치 로드

pgvector 활용

PostgreSQL의 pgvector 확장으로 1024차원 벡터를 저장하고, 코사인 거리 연산자(`<=>`)로 유사도 검색을 수행합니다. Hibernate Vector 라이브러리로 JPA Entity에서 `vector(1024)` 타입을 직접 사용합니다.

- **FloorplanAnalysis**: 도면의 자연어 설명을 Qwen3-Embedding-0.6B로 임베딩 → 유사 도면 검색
- **InternalEval**: 사내 평가 기준 문서를 임베딩 → RAG 검색
- **하이브리드 검색**: 벡터 유사도(0.8 가중치) + 텍스트 키워드 매칭(0.2 가중치)

API 설계

Spring Boot REST API — 4개 도메인, 24개 엔드포인트

AuthController (`/api/auth`) — 9개

Method	Path	설명
POST	/check-email	이메일 중복 검사
POST	/signup	회원가입 (BCrypt 해싱)
POST	/login	로그인 → JWT 발급 (rememberMe: 1h/24h)
POST	/refresh	토큰 갱신
GET	/me	현재 사용자 정보 조회
POST	/profile	프로필 수정 (이름, 전화번호)
POST	/change-password	비밀번호 변경
POST	/mailSend	OTP 이메일 발송 (6자리, 5분 만료, 30초 쓰로틀)
GET	/mailCheck	OTP 인증번호 확인

FloorPlanController (`/api/floorplan`) — 5개

Method	Path	설명
POST	/analyze	도면 분석 미리보기 (Python 연동)
POST	/save	도면 저장 (메타데이터 생성 + S3 + DB)
GET	/my	내 도면 목록 조회
GET	/{id}/detail	도면 상세 조회
GET	/{id}/image	도면 이미지 조회 (S3 302 리다이렉트)

ChatbotController (`/api/chatbot`) — 6개

Method	Path	설명
POST	/chat	챗봇 대화 (텍스트/이미지, Python orchestrate 연동)
POST	/sessionuser	내 채팅방 목록
POST	/roomhistory	채팅방 히스토리 조회
POST	/editroomname	채팅방 이름 수정
POST	/deleteroom	채팅방 삭제
POST	/deleteallrooms	전체 채팅방 삭제

AdminController (`/api/admin`) — 4개+

Method	Path	설명
GET	/stats	대시보드 통계 (6개 카드)
GET	/logs	활동 로그 (서버사이드 페이징 + 필터)
GET	/floorplans	전체 도면 목록 (서버사이드 페이징)
POST	/searchfloorplan	도면 검색 (이름/이메일/날짜/방 수 필터)
POST	/deleteentities	도면 일괄 삭제

Python FastAPI — 4개

Method	Path	설명
POST	/analyze	CV 파이프라인 + LLM 분석 (미리보기)
POST	/generate-metadata	13개 메트릭 + document + 임베딩 생성
POST	/orchestrate	의도 분류 + 에이전트 라우팅 (텍스트/이미지)
GET	/health	헬스 체크

기술적 도전과 해결

1. vLLM 토큰 최적화

문제: Qwen3-8B(max_model_len=8,192)에서 CV 분석 LLM이 7,801/8,192 토큰(95%)을 사용하여 응답 잘림이 빈번했습니다.

해결: max_model_len을 16,384로 확장하고, 시스템 프롬프트를 6,610→1,500 bytes로 경량화했습니다. `max_tokens=1,500` 설정과 FEW-SHOT 예시, 중단 규칙("절대 반복하지 마라")을 추가하여 무한 반복 생성을 방지했습니다.

결과: 토큰 사용률 95% → 48%로 개선, 응답 잘림 문제 해소

2. JSON 자동 복구 로직

문제: vLLM 토큰 초과로 응답이 중간에 잘렸을 때 JSON 파싱이 실패하여 전체 분석 결과를 사용할 수 없었습니다.

해결: 4단계 자동 복구 로직 — ① 열린 따옴표 닫기 → ② trailing comma 제거 → ③ 열린 괄호 닫기 → ④ 마지막 완전한 값까지 재시도

결과: 불완전한 응답에서도 Pydantic 모델 검증을 통과하는 유효한 JSON 복원

3. sLLM 출력 정제

문제: sLLM이 `(space_12)`, `(contains.windows 비어있음)`, `(edge)` 같은 내부 그래프 용어를 사용자 답변에 노출

해결: 정규식 후처리(`_clean_llm_output`)로 10가지 이상의 내부 용어 패턴 자동 제거. boolean 값(`true`/`false`)은 코드에서 한글(`존재`/`없음`)로 확정적 변환

4. 동시 요청 처리

문제: `async def` 엔드포인트에서 동기 블로킹 코드가 이벤트 루프를 블로킹하고, 단일 DB 커넥션 공유로 데이터 경합 발생

해결: 모든 엔드포인트를 `def`로 전환(FastAPI 스레드풀 자동 실행). DB를 `ThreadedConnectionPool(1,4)`로 교체하고, 참조 40곳 이상인 서비스는 `threading.local()` 기반 스레드별 독립 커넥션으로 해결

결과: 이미지 분석 3~4명, 텍스트 검색 8~10명 동시 지원

5. 검색 필터 점진적 완화

문제: 이미지 유사 도면 검색에서 정확한 필터 적용 시 결과가 0~1개만 나오는 경우 빈번

해결: 7단계 Progressive Relaxation — 전체 필터 → 욕실 수 제거 → Bay 수 제거 → 방 수 ±1 → 방 수 제거 → 구조 유형 제거 → 순수 벡터 유사도

결과: 모든 검색에서 유사 도면 3개 반환 보장

CI/CD 및 배포 인프라

GitHub Actions 자동 배포 파이프라인

main 브랜치 push 시 EC2에 SSH 접속하여 3개 서비스를 순차 배포합니다.

```
GitHub Push (main)
    ▼ SSH (appleboy/ssh-action)
    ├─ 1. git pull origin main
    ├─ 2. Python: pip install → systemctl restart skn20
    ├─ 3. Spring Boot: mvn package -DskipTests → systemctl restart springboot
    └─ 4. React: npm install → vite build → systemctl restart nginx
```

인프라 구성

서비스	리소스	역할
AWS EC2	Ubuntu	React + Spring Boot + FastAPI 호스팅
AWS RDS	PostgreSQL + pgvector	전체 데이터 저장 (9개 테이블)
AWS S3	arae-floorplan-images	도면 이미지 저장
RunPod Serverless	GPU 오토스케일링	CV 추론(4모델), 임베딩(Qwen3), 리랭킹(CrossEncoder)
RunPod Pod	A100 SXM 80GB	vLLM 서버 (Qwen3-8B + LoRA 어댑터)
Nginx	리버스 프록시	3개 서비스 라우팅 + 정적 파일 서빙
systemd	서비스 관리	Python/Java 프로세스 자동 재시작

RunPod GPU 구성

Serverless (초 단위 과금, 오토스케일링):

- CV 추론: YOLOv5×2(OBJ/OCR) + FCN(STR) + DeepLabV3+(SPA)
- 임베딩: Qwen3-Embedding-0.6B (1024차원)
- 리랭킹: CrossEncoder (ms-marco-MiniLM-L-6-v2)
- Docker 이미지에 모델 사전 다운로드로 콜드스타트 최소화

Pod (시간 단위 과금, 전용 인스턴스):

- vLLM 서빙: Qwen3-8B + LoRA 파인튜닝 어댑터, max_model_len=16,384
- EC2 ↔ RunPod SSH 터널 구성 (localhost:8888)

데이터 초기화

서버 시작 시 DataInitializer가 참조 데이터를 자동 로드합니다. 중복 방지(count > 0)와 JDBC batch insert(5,000건 단위)로 대량 데이터를 효율적으로 처리합니다.

1. InternalEval (JSON, 평가 기준 + 1024-dim 임베딩)

2. Usebuilding (CSV, 건축물용도), Law (CSV, 법규/조례), LandChar (CSV, 토지특성)
3. FloorPlan + FloorplanAnalysis (JSON, 기존 도면 + 메트릭 + 임베딩)

담당 영역

Frontend

- Feature-based 아키텍처 설계 (auth, chat, floor-plan, admin, profile 5개 모듈)
- JWT 자동 갱신 — 만료 5분 전 + 사용자 활동 감지, Axios request/response 인터셉터
- 토큰 저장소 분기 — rememberMe: localStorage(24h) / sessionStorage(브라우저 종료 시 삭제)
- 마커 기반 답변 파싱 — `[도면 #N]`, `[유사 도면 #N]` 정규식 → 요약 + 썸네일 + ImageModal
- 5단계 회원가입 (이메일 OTP 인증), 비밀번호 재설정
- 관리자 대시보드 (통계 카드, 도면 관리, 활동 로그, 서버사이드 페이지네이션)

Backend

- REST API 설계 — Auth, FloorPlan, Chatbot, Admin 4개 도메인, 24개 엔드포인트
- JWT 인증 — HS256, rememberMe(24h/1h), Spring Security 필터 체인
- Python FastAPI 연동 — RestTemplate, multipart/form-data, 10분 타임아웃
- S3 이미지 관리 — 업로드/조회(302 리다이렉트)/삭제(cascade)
- 대용량 배치 로드 — JDBC batch insert(5,000건), @Transactional 원자성
- OTP 이메일 — JavaMailSender, 6자리 OTP, 5분 만료, 30초 쓰로틀
- Entity 설계 — 9개 Entity, cascade ALL + orphanRemoval, Lazy Loading

AI Server

- Orchestrator 패턴 — 의도 분류(GPT-4o-mini) → 에이전트 라우팅 + Fallback
- CV 4단계 파이프라인 → Shapely 기하학 → 토폴로지 그래프 생성
- RAG — pgvector + 사내 평가 기준 + LLM 구조화 출력 + Pydantic 검증
- sLLM 프롬프트 엔지니어링 — 경량화(6,610→1,500B), FEW-SHOT, 중단 규칙, 정규식 후처리
- 하이브리드 검색 (벡터 0.8 + 텍스트 0.2) + Cross-encoder 리랭킹 + LRU 캐시(500개)
- 필터 점진적 완화 7단계 + JSON 자동 복구 4단계 + 토큰 로깅
- 동시성 — async→def 전환, ThreadedConnectionPool, threading.local()

Infra

- GitHub Actions CI/CD (main push → 3개 서비스 순차 자동 배포)
- RunPod Pod (A100 SXM 80GB) vLLM 관리 + RunPod Serverless 오토스케일링
- EC2 ↔ RunPod SSH 터널 + Nginx 리버스 프록시 + systemd 서비스 관리

성과

항목	수치
CV 파이프라인	4개 모델 통합 (YOLOv5×2 + FCN + DeepLabV3+) → 토폴로지 그래프 자동 생성
건축 메트릭	13개 자동 추출 + 1024차원 임베딩 (Qwen3-Embedding-0.6B)
sLLM 토큰	95% → 48% 사용률 개선 (max_model_len 8K→16K + 프롬프트 경량화)
검색 안정성	필터 7단계 점진적 완화로 유사 도면 3개 반환 보장
API 규모	Spring Boot 24개 + FastAPI 4개 = 28개 엔드포인트
DB 설계	9개 Entity, pgvector 1024-dim 벡터 검색, JDBC 배치 로드
동시 처리	이미지 분석 3~4명, 텍스트 검색 8~10명 동시 지원
배포 자동화	GitHub Actions → EC2 SSH → 3개 서비스 순차 자동 배포

회고

잘한 점

- **Orchestrator 패턴 도입**: 의도 분류 → 에이전트 라우팅 구조로 도면 검색, 법규 검색, 이미지 분석을 하나의 챗봇 인터페이스로 통합
- **마커 기반 파싱**: LLM이 자유롭게 마크다운을 생성하되, `[도면 #N]` 마커로 프론트엔드와 약속된 구조를 유지하여 유연성과 파싱 안정성을 동시에 확보
- **점진적 완화 알고리즘**: 엄격한 필터에서 시작하여 단계적으로 완화함으로써, 정확도와 검색 결과 보장을 균형 있게 달성
- **풀스택 경험**: Frontend부터 GPU 서버 관리까지 전 계층을 직접 설계·구현하여 시스템 전체를 이해하고 병목을 빠르게 진단

아쉬운 점

- **WebSocket 미도입**: 현재 챗봇은 HTTP 폴링 방식으로, 분석 진행률을 실시간으로 보여주지 못함
- **테스트 코드 부족**: CI/CD에서 테스트를 스킵(-DskipTests)하고 있어, 자동화된 품질 검증이 부족
- **보안 설정 하드코딩**: API 키, DB 비밀번호 등이 설정 파일에 평문 저장 → AWS Secrets Manager 개선 필요

추후 개선 방향

- WebSocket 기반 실시간 분석 진행률 표시
- React Query/SWR 도입으로 클라이언트 캐싱 최적화
- 단위/통합 테스트 추가 및 CI 파이프라인 통합